SPL COMPILER (LEXER, PARSER, SEMANTICS, CODE GENERATION)

User Manual

Group 7

MEMBERS

Student number	Name	Role
u23533413	Zamokuhle Zwane	Speaker. Part 3b (functions and inlining)
u23533596	Diya Narotam	Part 2a (names and scopes)
u23534943	Mpumelelo Njamela	Part 3a (code generation for algorithms)
u23534975	David Kalu	Part 2b (type analysis), tests

The executable is the file **group-7.jar**. It is already compiled; nothing has to be compiled by the marker. One program does everything: it reads an SPL source file, checks it (lexer, parser, scope and name analysis, type analysis) and, if the program is correct, writes an executable C++ file. This submission is a **1st Class** submission (see the declarations at the end).

STEP 1: CHECK THAT JAVA IS INSTALLED

Open a terminal (we used WSL) and run:

```
java -version
```

Java 11 or newer is required.

STEP 2: PREPARE A FOLDER

Create an empty folder, place **group-7.jar** in it and the SPL test file next to it (in the examples it is called SPL.txt, any name works). Open the terminal in that folder.

STEP 3: RUN THE COMPILER

```
java -jar group-7.jar SPL.txt
```

Optionally a second argument names the output file: `java -jar group-7.jar SPL.txt result.cpp`. Without arguments the compiler reads **SPL.txt** and writes **output.cpp** in the current folder.

STEP 4: READ THE RESULT

A) The SPL program is correct. The screen shows one line and nothing is written to the error stream:

```
OK: 'SPL.txt' compiled to 'output.cpp' (all numbers are int).
```

These files are written in the current folder:

- **output.cpp**: the generated executable C++ program (the code mine for the functions is in one big comment at the top, followed by the executable `main`).
- **tree.xml**: the syntax tree from the parser (as in Phase 1).

- **symtable.txt**: the symbol table: every declared name with its unique system-generated name (sys1, sys2, ...), kind, scope level, and, from the type analysis, its type; plus the program type (int or float).
- **tree_renamed.xml**: the syntax tree in which every user-defined name is replaced by its unique system-generated name.

B) The SPL program is not correct. Exactly one error message is printed on the error stream, with line and column (the one exception is the float-integer conflict rule, which has the fixed text required by the specification). No output file is written, and an old output file from a previous run is removed.

The exit code can be checked (Linux, macOS, WSL: `echo $?`, Windows: `echo %ERRORLEVEL%`):

Exit code	Meaning
0	valid program, output file written
1	lexical, syntax, naming/scope or type error in the SPL program
2	input file cannot be read, output cannot be written, or the program is too large to compile (see Limits)

STEP 5: COMPILE AND RUN THE GENERATED PROGRAM

```
g++ output.cpp -o program
./program
```

The program prints what the SPL program prints: a `print` statement writes its string or number with `std::cout` and, as the specification says, adds no separator or line break.

WHAT THE COMPILER CHECKS

Names and scopes (Part 2a). A variable needs a declaration at the same or an outer scope level; the nearest declaration wins; no double declarations on one level; local variables of a function may not mask its parameters; a called function must be declared in the F_DECL of the same scope level as the calling ALGO (this rules out recursion, calls to sibling or outer functions and calls to functions that are only visible somewhere else); function names on one level are unique (no overloading). Every declaration gets a unique system-generated name, which is stored in the symbol table and used by the code generator.

Types (Part 2b). Variables are numeric. A call used as an instruction must be a void procedure; a call used inside a term must be a num function; conditions must be boolean; arguments and operands must be numeric. A program that contains both a `mod` and a `div` node anywhere is rejected at once with the message *A float-integer-conflict might perhaps be possible*; if it contains `mod`, no number may contain a decimal point. The whole program is float if it contains `div` or a decimal number, otherwise int, and every declaration in the generated C++ uses that type. Two further checks that the generated code needs: a call must have as many arguments as the function has parameters, and in an int program every number must fit into a 32-bit int.

Code generation (Parts 3a and 3b). Booleans become jumping code (no and/or/not and no else in the target); if/else, all four loop forms, terms, assignments and printing follow the specification sheet. Functions are translated by **inlining**: every function body is kept in the code mine and copied into the place of each call, with fresh labels for every copy, so no run-time stack is needed. Nested functions are inlined into their parent inside the mine first.

EXAMPLES OF ERROR MESSAGES

```
Semantic error at line 1, column 5: the variable '#x' is used here but is not declared
in any V_DECL at scope level 0 or in any enclosing scope above it.

Semantic error at line 1, column 19: the local variable '#a' masks the parameter '#a'
of the same function (declared at line 1, column 12).

Type error at line 1, column 37: the function '#f' returns a number, but a call used as
an instruction must be a void procedure (use its value in a term, e.g. an assignment,
instead).

A float-integer-conflict might perhaps be possible
```

DECISIONS WHERE THE SPECIFICATION LEAVES ROOM

- **Parameter passing:** the sheet writes `p1 = a1`. We generate `a1 = p1` (the parameter receives the value of the captured argument), which is what makes a call by value work.
- **and / not:** swapping the two labels alone would make *and* behave like *or*, so the translation of not, and and or each add the jump (or the intermediate label) that makes both outcomes reach the right place. Verified by a test that checks all 312 boolean expressions up to depth two against an independent evaluator. *and* and *or* evaluate their second operand only when needed.
- **Code mine:** C does not nest comments, so inside the big comment the small comments are written as // comments.
- **Variables** start with the value 0 (`typ x; x = 0;`); declarations never carry an initializer, because C++ forbids a goto that jumps over an initialization.
- **Labels** have the form L1, L2, ...; each inlined copy of a function body gets a complete new set.

Limits. Inlining copies a body once per call, so deeply nested functions that each call their sub-function many times multiply out. If a single inlined copy would exceed 16 million characters the compiler stops with an error message instead of running out of memory. As in C, integer overflow, `mod` by zero and floating-point overflow in the generated program are not checked at run time.

DECLARATIONS

Declaration 1: submission class. We, the members of Group 7, declare that this submission is made in the **1st Class** category (maximum 6/6 points: Parts 2a, 2b, 3a and 3b, including the generation of executable target code) and not in the 2nd Class category. All members of the group agree with this declaration.

Student number	Name	Signature	Date
u23533413	Zamokuhle Zwane		
u23533596	Diya Narotam		
u23534943	Mpumelelo Njamela		
u23534975	David Kalu		

Declaration 2: knowledge of the ZIP file. We, the members of Group 7, declare that every member is fully aware of the whole contents of the ZIP file (group-7.pdf, this user manual, and group-7.jar, the executable) and agrees with the submission of the ZIP file in this form.

Student number	Name	Signature	Date
u23533413	Zamokuhle Zwane		
u23533596	Diya Narotam		
u23534943	Mpumelelo Njamela		
u23534975	David Kalu		

Thank you for marking our submission.